



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

ELEKTRO- UND
INFORMATIONSTECHNIK

PRAKTIKUM INFORMATIK 1

M. Niemetz, M. Schumm

Pointer, Zeichenketten, Dateien

Pointer-Experiment und Kryptografie

November 2025, Version: 1.55

Fakultät Elektro- und Informationstechnik

Einleitung

Der erste Teil dieser Aufgabe soll Sie durch Experimente mit Pointern vertraut machen. Hier geht es weniger darum, ein Anwendungsprogramm zu entwickeln, das eine bestimmte Funktion realisiert, als vielmehr um Erkenntnisgewinn bezüglich der Möglichkeiten, die der Pointermechanismus bietet.

Wichtig: Nur weil ein Experiment funktioniert, sind Sie noch nicht fertig, denn vielmehr gilt es aus jeder Teilaufgabe ein Verständnis für die internen Abläufe hinter den Pointer-Operationen zu entwickeln.

Hinweis: Um den Lernerfolg zu sichern, ist es bei dieser Aufgabe eine besonders wichtige Idee, sich während der Experimente Notizen zu machen, so wie Sie sich das bereits bei den vorangehenden Aufgaben angewöhnt haben.

Pointer sind die Umsetzung der Sprache C für einen indirekten Zugriff. Das heißt, ein Pointer ist eine Variable, welche den Ort enthält, an der der gewünschte Wert im Speicher abgelegt ist. Pointer speichern also Adressen im Arbeitsspeicher des Rechners.

Hinweis: Diese Vorgehensweise, Werte zu adressieren ist ein sehr wichtiges Gestaltungswerkzeug. Von Assembler bis hin zu modernen Hochsprachen findet diese Methode Anwendung. Allerdings wird in manchen Programmiersprachen (z.B. Python) dieser indirekte Zugriff sehr konsequent versteckt und nur die Auswirkungen bleiben sichtbar.

Pointer werden deklariert und definiert wie andere Variablen auch, allerdings wird bei der Typdefinition jeweils das Zeichen `*` an den Typpnamen angehängt (z.B. `int*` für eine Pointervariable, welche auf eine Adresse verweist, die einen `int`-Wert enthält).

Ein Zugriff auf die Pointervariable liefert bzw. verändert daher die im Pointer gespeicherte **Adresse**. Soll hingegen ein Zugriff auf die **Daten** an der Adresse erfolgen, auf die der Pointer verweist, so muss der Pointer „dereferenziert“ werden. Hierfür zuständig ist der Dereferenzierungsoperator `*`, der das selbe Operatorzeichen verwendet wie die Multiplikation aber im Gegensatz dazu nur einen (rechten) Operanden besitzt.

Wichtig: Beachten Sie, wie viele verschiedene Bedeutungen das Zeichen `*` im C-Quelltext erfüllt:

- Multiplikationsoperator wenn es mit zwei Operanden auftritt;
- Dereferenzierungsoperator wenn es mit einem Operanden auftritt;
- Pointer-Deklarator wenn es in einer Deklaration bei einem Typ auftritt;

Diese Vieldeutigkeit tritt bei vielen Symbolen wie zum Beispiel +, () oder & auf und führt zu Verwirrung, wenn man versucht C-Code intuitiv zu verstehen ohne diese Kontextsensitivität der Sprache verstanden zu haben.

Soll ein Pointer einmal auf „nichts“ zeigen, so muss er mit dem Wert `NULL` belegt werden. Dieser Wert für den Inhalt des Pointers bedeutet vereinbarungsgemäß, dass der Pointer „nirgendwohin“ zeigt und daher auch nicht dereferenziert werden darf.

Natürlich kann diese Kurzerklärung keine detaillierte Recherche ersetzen. Also lesen Sie am besten in Ihrem persönlichen Lieblings-Lehrbuch nach.

Recherche ☕☕

Lesen Sie im Skript oder Buch nach, wie Pointer funktionieren

Vorbereitung

- ☐ ☕☕ **Lesen Sie im Skript oder Buch nach, wie Pointer funktionieren:** Sie benötigen in dieser Aufgabe das Definieren von Pointervariablen sowie den Zugriff auf die vom Pointer referenzierten Daten (Dereferenzierung). Schreiben Sie Ihre Erkenntnisse auf und verdeutlichen Sie sich die Zusammenhänge mit einer kleinen Zeichnung! 3

1 Testumgebung

Für die Durchführung der Versuche benötigen Sie eine kleine Testumgebung. Sie besteht aus der Deklaration der folgenden Variablen:

- Ganzzahlvariable `i` vom Datentyp `int` initialisiert mit 0
- Pointervariable `pi` vom Datentyp Pointer auf `int` und initialisiert mit `NULL`
- Pointervariable `ps` vom Datentyp Pointer auf `short` und initialisiert mit `NULL`
- Pointervariable `pc` vom Datentyp Pointer auf `char` und initialisiert mit `NULL`
- Array `vi` aus `int`-Größen mit der Vorbelegung 0, 1, 2, 3
- Array `vc` aus `char`-Größen mit der Vorbelegung durch die Buchstaben "`abcdefghijklmno`".

Hinweis: Sie können das letztgenannte Array `vc` erzeugen durch:

```
char vc[]="abcdefghijklmno";
```

Am Ende des Programmes fügen Sie noch die folgende Zeile ein:

```
printf("\nENDE\n");
```



Erstellen Sie Testvariablen für Ihre Experimente.

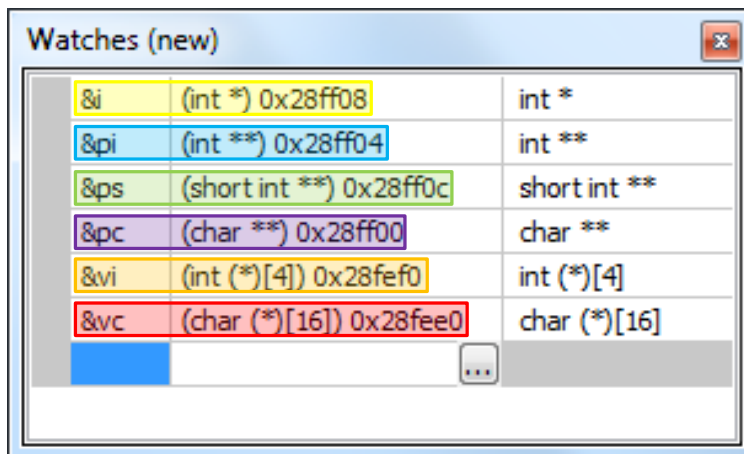


Abbildung 1: Watches-Fenster Code::Blocks mit zusätzlichen farblichen Hervorhebungen. Gut zu erkennen ist, dass nicht nur Variablennamen im Watches-Fenster eingegeben werden können, sondern auch der Adressoperator Verwendung finden kann. Dieser wird hier intensiv eingesetzt. Die farblichen Hervorhebungen wurden in der Anleitung manuell hinzugefügt. CodeBlocks zeigt diese nicht an.

Setzen Sie einen **Breakpoint** für den Debugger auf diese zuletzt eingelegte Zeile.

Wichtig: Kopieren Sie Ihre Testumgebung in eine zweite Datei, so dass Sie bei Bedarf wieder zur Ausgangssituation zurückkehren können.

Nun prüfen Sie Ihre Testumgebung mit dem Debugger. Zu diesem Zweck sollen Sie die Adressen der verwendeten Variablen mit Hilfe des Watches-Fensters des Debuggers anzeigen lassen.

Hinweis: Sie können im Watches-Fenster zusätzlich zu den automatisch angezeigten Werten auch andere Dinge anzeigen lassen. Hierfür müssen Sie diese manuell ergänzen. Dabei können Sie nicht nur Variablennamen eingeben, sondern können auch einfache Ausdrücke verwenden. Besonders in diesem Zusammenhang ist der Adressoperator, welcher es Ihnen gestattet, die Adresse einer Variable zu ermitteln und anzeigen zu lassen, wie in Abbildung 1 dargestellt ist.

Ein entsprechendes Beispiel ist Abbildung 1 zu entnehmen.

Hinweis: Vermutlich verwendet Ihre Testumgebung andere Adressen für die Variablen. Dies ist völlig normal.

Um den Inhalt der Variablen betrachten zu können, verwenden Sie dieses Mal die Memory-Dump-Funktion des Debuggers, die Sie vom Semesteranfang bereits kennen. Zur Anzeige aller Variablen gleichzeitig müssen Sie hierzu die bezüglich des Wertes niedrigste Adresse bestimmen und diese in das Adressenfeld des Memory-Dump-Fensters eintragen. Sie finden dann die übrigen Variablen im dargestellten Speicherausschnitt.



Erstellen Sie ein Watches-Fenster zur Prüfung der Testumgebung.

...

Erstellen Sie einen Memory-Dump um die Werte im Computerspeicher betrachten zu können.

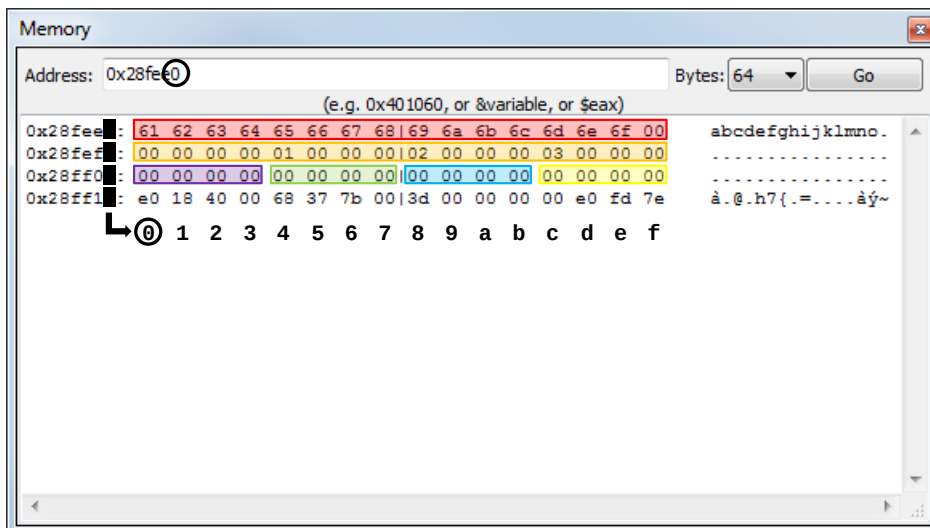


Abbildung 2: Memory Dump in Code::Blocks: Die farblich gekennzeichneten Bereiche entsprechen dabei den Variablen in Abbildung 1. Die Adresse zu Beginn der Zeile stellt den Speicherort des ersten Wertes in der Reihe dar. Die übrigen Werte in der Zeile liegen dann an aufsteigend aufeinanderfolgenden Speicheradressen.

In der Abbildung 2 sind die zu den Beispielen aus der Abbildung 1 gehörenden Memory Dumps zu sehen. Die zu den Variablen gehörenden Speicherbereiche sind farblich gekennzeichnet. Diese farbliche Kennzeichnung finden Sie in der IDE nicht.

Die Hexadezimalzahl zu Beginn jeder Zeile stellt die Adresse des ersten Werts dar, der dann rechts neben der Adresse dargestellt wird. Für die weiteren Werte in der Zeile ergeben sich die Adressen, indem Sie die letzte (und evtl. auch vorletzte) Stelle, wie in den Abbildungen angedeutet, durch die entsprechen erhöhten Ziffernwerte ersetzen.

Wichtig: Die Adressen bei der Durchführung der Experimente müssen nicht mit den Beispielen in den Abbildungen übereinstimmen. Sie können sich auch von Experiment zu Experiment und von Programmstart zu Programmstart ändern!

Wichtig: Weitere Informationen zu der Verwendung des Debuggers und der Erstellung eines Memory-Dumps entnehmen Sie bitte der Debugger Anleitung, welche im E-Learning-Kurs zu finden ist und die Ihnen noch von den ersten Aufgaben her bekannt sein sollte.

Erstellen Sie einen entsprechenden Memory-Dump und bestimmen Sie in diesem die Positionen der Variablen wie im Beispiel in Abbildung 2.

Frage: Wie hängen die verschiedenen Definitionen der Variablen der Testumgebung mit den Verhältnissen im Rechnerspeicher zusammen?

Frage: Wie muss man sich die Anordnung im Speicher vorstellen?



Identifizieren Sie die Position der Variablen Ihrer Testumgebung im Memory-Dump.

Frage: Versuchen Sie sich die Verhältnisse anhand einer Skizze bewusst zu machen.

Hinweis: Verändern Sie vorübergehend die Initialisierungswerte der Variablen, um die Speicherzuordnung zu den Variablen zu verifizieren.

2 Experimente

2.1 Byte-Reihenfolge, Endianness

Wenn die CPU eine `int` Größe speichert, sind mehrere (typisch vier¹) Bytes abzulegen.

Die Reihenfolge der Inhalte in der Speicherzellen spielt in der Praxis nur selten eine Rolle, denn wenn die verschiedenen Bytes beim späteren Lesen wieder in der gleichen Reihenfolge geholt und zusammen gesetzt werden, erhält man wieder den ursprünglichen Wert.

Es ist die Aufgabe des Compilers, die Lese- und Schreibzugriffe konsistent ausführen zu lassen. Relevant ist der Unterschied nur, wenn man am Variablenzugriff „vorbei“ auf die einzelnen Speicherstellen zugreift - oder die Speicherstellen eine Hardware-bedingte Bedeutung besitzen (wie zum Beispiel die Bytes in einem Displayspeicher oder einem Netzwerkstream).

Die Reihenfolge, in der die zusammengehörigen Bytes im Speicher abgelegt werden, ist daher prinzipiell beliebig. Daher nutzen unterschiedliche Hersteller bzw. unterschiedliche Prozessor- und Mikrocontrollerfamilien unterschiedliche Reihenfolgen. Es haben sich zwei Varianten eingebürgert:

Big Endian Das Byte mit den höchstwertigen Bits wird als erstes unter der Anfangsadresse der `int` Größe abgelegt. Das Byte, das das Bit mit Stellenwert 1 enthält, kommt als letztes (hat also die höchste Adresse im Speicher). Das „Ende“ der Zahl ist also bei der großen Adresse zu finden.

Little Endian Das Byte mit den niedrigstwertigen Bits wird als erstes unter der Anfangsadresse der `int` Größe abgelegt. Das Byte mit den höchstwertigen Bits kommt als letztes. Damit ist hier das „Ende“ der Zahl bei der kleinsten Adresse zu finden.



Ermitteln Sie, welche „Endianness“ vom Prozessor in Ihrem PC verwendet wird.

¹Dies gilt für einen Compiler, der für `int`-Variablen 32 Bit verwendet. Es gibt auch Compiler, welche `int`-Daten mit zwei Byte (16 Bit) oder acht Byte (64 Bit) ablegen und häufig lässt sich dies auch durch Compileroptionen umschalten. Grundsätzlich sind alle diese Größen von `int` standardkonform. Verlassen Sie sich also nicht auf eine bestimmte Größe von `int`, denn garantiert wird nur eine Mindestgröße von 16 Bit. Sie können die Größe von Variablen im Speicher jederzeit mit Hilfe des Operators `sizeof` bestimmen.

Frage: Welche Variante hat Ihr Rechner in seiner CPU realisiert?

Gehen Sie zur Klärung der Frage in folgenden Schritten vor:

- Zum Test brauchen Sie als Wert der Variable ein Bitmuster, das die Reihenfolge der Bytes erkennen lässt. Weisen Sie der Variable `i` zum Beispiel einen ganzzahligen Wert zu, dessen 4 Bytes die Werte 4, 3, 2, 1 haben. Das Byte mit dem höchsten Stellenwert soll den Wert 4 haben, usw. bis zum Byte mit dem niedrigsten Stellenwert, das den Wert 1 haben soll.

Hinweis: Benutzen Sie auf der rechten Seite der Wertzuweisung eine Literalkonstante, die hexadezimal angegeben ist, dann ist das gewünschte Ergebnis einfacher zu erreichen!

- Überprüfen Sie mit Hilfe des Memory-Dumps die Reihenfolge der Bytes von `i` im Speicher! Stellen Sie fest, welche Endianness die CPU Ihres PCs verwendet!

2.2 Operatoren für Adressen bzw. Pointer, Formatelement für Pointer

Mit Hilfe des Adressoperators `&` ist es möglich, die Adresse seines (rechten) Operanden zu ermitteln. Diese besitzt dann den Typ *Pointer auf den Typ des rechten Operanden*.

Diesen Wert können Sie zum Beispiel in einer Pointer-Variable passenden Typs speichern.

- Weisen Sie dem Pointer `pi` die Adresse der Variable `i` zu. Geben Sie jeweils die **Adresse** und den **Inhalt** der Variablen `i` und der Pointer-Variablen `pi` auf die Konsole aus.

Hinweis: Mit dem Formatelement `%p` im Formatstring von `printf` erhält man eine Ausgabe, wie sie für Adressen geeignet ist (typischerweise eine Hexadezimalzahl).

Vergleichen Sie Ihre Ausgaben mit dem Memory-Dump.

Machen Sie sich (auf Papier oder mit einer elektronischen Zeichenmöglichkeit) **graphisch** die Beziehungen zwischen

- dem Wert von `i` und der Adresse von `i`
- dem Wert der Pointervariablen `pi` und deren Adresse
- der Adresse von `i` und dem Wert der Pointervariablen `pi`

klar.

Hinweis: Entwickeln Sie eine graphische Darstellung für den Zusammenhang zwischen Variable, Adresse, Pointer und des vom Pointer referenzierten Werts, anhand derer Sie jemand anders (z.B. ihren Kommilitonen) diese Zusammenhänge erklären können.



Arbeiten Sie mit dem Adressoperator:

Frage: Stellen Sie sicher, dass Sie diese Zusammenhänge bei der Abgabe erklären können. Im Idealfall bringen Sie Ihre Zeichnung mit zur Abgabe, das erleichtert Ihnen das Erklären!

2.3 Pointertyp und Zugriff auf die Bezugsvariable

Zu Testzwecken führen Sie als nächstes an Ihrem Programm Änderungen durch, die in einer realen Programmsituation keinen Sinn ergeben würden, bzw. schwere Programmierfehler wären. Das Ziel ist, dadurch besser zu verstehen, wie sich solche Fehler auswirken.

Testen Sie zunächst, wie die Kompilation läuft, wenn Sie die Adresse von `i` den Variablen `ps` bzw. `pc` zuzuweisen versuchen.

Frage: Wie lautet die Fehlermeldung oder Warnung des Compilers? Was bedeutet diese? Wo liegt das Problem, auf das der Compiler hinweisen will?

Die obigen Fehlermeldungen oder Warnungen des Compilers können umgangen werden, indem die Zuweisung explizit erzwungen wird:

Mit einer Typcast-Operation kann man den *Typ* einer Variable temporär für den aktuellen Lesezugriff umwandeln, ohne dass sich der enthaltene Wert im Speicher (bei Pointern also die Zieladresse) ändert. Dazu wird einem Ausdruck der gewünschte Zieltyp in runden Klammern vorangestellt. Also z.B.

```
my_char_pointer = (char*)&my_int_variable;
```

Bringen Sie die Fehlermeldung oder Warnung des Compilers bei obigem Experiment zum Verschwinden, indem Sie für `ps` und `pc` jeweils eine geeignete Typumwandlung einfügen!

Jetzt haben Sie vier Möglichkeiten, auf den Wert von `i` zuzugreifen:

- über den Namen der Variablen `i`
- über den Pointer `pi` auf `int`
- über den Pointer `ps` auf `short`
- über den Pointer `pc` auf `char`

Die ersten beiden Möglichkeiten haben Sie bereits genutzt. Jetzt greifen Sie über die Pointer mit dem „falschen“ Typ zu und beobachten, was passiert:

In `i` sollte aus den vergangenen Teilaufgaben noch der Wert stehen, der den Bytes 4, 3, 2, 1 entspricht. Falls hier ein anderer Wert steht, korrigieren Sie dies bitte.

Weisen Sie der vom Pointer `ps` referenzierten Größe unter Benutzung des Dereferenzierungs-Operators einen neuen Wert zu:



Verdeutlichen Sie sich die Bedeutung des Pointertyps für den Zugriff.

Um den Test zu erleichtern, verwenden Sie als neuen Wert die Zahl, deren höherwertiges Byte 44_{16} entspricht und deren niederwertiges Byte 33_{16} lautet.² Der Wert soll als Literalkonstante hexadezimal angegeben werden.

Geben Sie den neuen Inhalt der Variablen `i` aus und vergleichen Sie ihn mit dem Memory-Dump.

Frage: Interpretieren Sie die Ausgabe! Wie ist die Reihenfolge der Bytes für Short-Größen im Speicher?

Weisen Sie dann der vom Pointer `pc` referenzierten Größe unter Benutzung des Dereferenzierungsoperators einen neuen Wert zu!

Verwenden Sie als Wert den ASCII Code des Zeichens X. Er soll als `char`-Literalkonstante in einfachen Anführungszeichen angegeben werden.

Geben Sie den Inhalt der Variablen `i` aus und vergleichen Sie ihn mit dem Memory-Dump.

Frage: Interpretieren Sie die Ausgabe! Was passiert beim Zugriff mit Pointern unterschiedlichen Typs auf die selbe Speicheradresse?

Machen Sie sich bewusst, dass die Pointer `ps` und `pc` als Wert die gleiche Adresse haben, nämlich die Adresse von `i`. Die unterschiedliche Breite des Zugriffs wird nur durch den Typ des jeweiligen Pointers bestimmt, den der Compiler während des Übersetzungsvorganges „im Kopf“ hat.

Nachdem Sie nun wissen, was in solchen Fällen passiert, hier noch die längst überfällige Warnung:

Wichtig: Machen Sie die Dinge in diesem Abschnitt **nie wieder** nach. Beim unsachgemäßen Umgang mit Pointern ~~kann~~ **kommt** es zu unvorhersehbarem Fehlverhalten Ihres Programmes ~~kommen~~. Achten Sie daher unbedingt immer darauf, nur mit korrekten Pointertypen auf Daten zuzugreifen.

Informieren Sie auch Ihre Kommilitonen (und später auch Ihre Kollegen), die diesen Hinweis nicht lesen konnten oder vergessen haben, über die Gefahr!

2.4 Pointer und Arrays, Pointerarithmetik

Die Implementierung der Array-Datentypen in C ist sehr schlank. Böseartige Stimmen behaupten, es gibt sie gar nicht. Dies rührt daher, dass Arrays vom C-Compiler abgesehen von der Definition weitestgehend wie konstante (deren Inhalt, also die in der Pointer-Variable gespeicherte Adresse, sich nicht ändern kann) Pointer behandelt werden.



Machen Sie sich die Äquivalenz von Array- und Pointerschreibweise klar. Experimentieren Sie mit Pointerarithmetik.

²Die Schreibweise 44_{16} gibt an, dass der gewünschte Wert in der Hexadezimaldarstellung 44 lautet.

Daher gilt zum Beispiel für unsere Variablen in der Testumgebung folgende Äquivalenz:

$$vi[i] \iff *(vi + i) \iff *(i + vi) \iff i[vi]$$

Dabei wird die Addition von `int` und einem Pointer so implementiert, dass der Pointer um Vielfache der Größe der referenzierten Größe im Speicher (also bei einem 32-Bit-`int` zum Beispiel typischerweise vier Byte) erhöht wird. Entsprechendes gilt auch für Inkrement- und Dekrementoperatoren.

Geben Sie die Adressen von `vi` und `vc` aus.

Frage: Muss man dazu den Adressbildungs-Operator auf den Namen des Arrays anwenden, oder genügt der Name des Arrays alleine? Testen Sie Ihre Hypothese!

Fügen Sie Wertzuweisungen ein, die der `int`-Pointervariablen `pi` die Adresse der ersten Komponente von `vi` zuweisen bzw. der `char`-Pointervariablen `pc` die Adresse der ersten Komponente von `vc`!

Geben Sie die Werte der Pointer aus und vergleichen Sie mit dem Memory-Dump!

Überzeugen Sie sich davon, dass es egal ist, ob man bei der Zuweisung z.B. `&vc[0]` schreibt oder nur `vc`.

Inkrementieren Sie die beiden Pointer z.B. mit `...+=1` oder `...++` und geben Sie die in den Pointern enthaltenen Adressen und die referenzierten Werte der Pointer aus!

Vergleichen Sie jeweils den alten und den aktuellen Wert von `pi` und `pc`!

Machen Sie sich bewusst, dass zwar die zahlenmäßigen Änderungen der beiden Pointer verschieden sind, aber beide Pointer jetzt jeweils die Adresse der nächsten Komponente ihres Arrays enthalten!

Hinweis: Visualisieren Sie den soeben experimentell ermittelten Sachverhalt in einer Zeichnung!

2.4.1 Kleine Rätselaufgabe

Machen Sie eine Voraussage, was der Ausdruck

```
*((char*)((int*)pc)+1))
```

liefert und testen Sie Ihre Vorhersage!

Hinweis: Diese und ähnliche Situationen meistert man „ganz einfach“ dadurch, dass man sich konsequent Schritt für Schritt „von innen nach außen“ durch den Ausdruck durcharbeitet.

Mit der Strategie „scharf anschauen und dann raten“ kommt man in der Regel nicht weiter.

2.5 Pointer als Unterprogramm-Parameter



Erkunden Sie die Funktionsweise der Pointer als Parameter bei Funktionen.

Pointer sind in C für viele Zwecke nützlich, je, unverzichtbar. In der vorangehenden Aufgabe haben Sie bereits das Problem der Rückgabe mehrerer Werte aus einer Funktion kennengelernt. Dort wurde es so gelöst, dass die Werte gemeinsam eine Strukturvariable gepackt wurden.

Diese Vorgehensweise ist nicht immer optimal, zum Beispiel, wenn die Werte nicht zusammengehören oder wenn ein Wert von der Funktion gelesen *und* modifiziert werden muss. In diesen Fällen ist das Ein- und vor allem das Auspacken der Werte sehr umständlich.

Daher wird häufig die Methode der Pointer-Parameter angewendet, welche die beschriebenen Nachteile nicht hat, aber dennoch die Kommunikation mehrerer Werte von der Funktion an den aufrufenden Programmteil erlaubt.

Die beiden Funktionen `set4711` und `set4712` befinden sich in der Datei `functions.c` im E-Learning-Kurs. Für die folgenden Experimente kopieren Sie bitte den gesamten Inhalt dieser Datei in Ihre C-Datei **vor** die `main`-Funktion.

`set4711` ist ein Unterprogramm, das einen Wert zurückliefert, indem es den Parameter beeinflusst.

Rufen Sie `set4711` mit der Adresse von `i` auf und geben Sie anschließend den Wert von `i` aus!

Verifizieren Sie, dass die Adresse von `i` als Ziel für die Wertzuweisung benutzt wird!

Rufen Sie `set4712` mit `i` auf und geben Sie anschließend den Wert von `i` aus! Überzeugen Sie sich davon, dass `set4712` nicht funktioniert!

Verifizieren Sie, dass nicht die Adresse von `i` als Ziel für die Wertzuweisung benutzt wird!

Weisen Sie die Aussage:

Innerhalb von `set4712` wird eine Variable gesetzt, deren Wert anfänglich gleich dem Wert von `i` ist, deren Adresse aber nicht mit der von `i` übereinstimmt.

nach, indem Sie die Funktion `set4712` im Debugger schrittweise abarbeiten lassen.

Beim Aufruf eines Unterprogramms mit einer Variablen als Parameter wird der Wert der Variablen auf den Stack³ kopiert. Mit dieser Kopie arbeitet das Unterprogramm. Beim Aufruf von `set4712` wird `i` deshalb nicht verändert, sondern nur eine temporäre Kopie auf dem Stack, die später **nicht** mehr in die beim Aufruf verwendete Variable zurückkopiert wird. Dadurch wird es auch möglich, der Funktion eine Konstante zu übergeben, die man nicht gar nicht beschreiben könnte.

Wird stattdessen ein Pointer übergeben, so wird zwar ebenfalls eine Kopie des Pointers auf dem Stack erzeugt. Aber das Unterprogramm kann über diese Kopie der Adresse auf die Bezugsvariable des Pointers zugreifen, in der der eigentliche Wert gespeichert ist, und diesen somit auch ändern. Daher funktioniert `set4711` und deswegen muss man übrigens auch an `scanf` die Adressen der einzulesenden Variablen übergeben.

Erzeugen Sie zwei verschiedene Skizzen für die Übergabe per Wert (also ohne Pointer, „call-by-value“) und für die Übergabe per Pointer („call-by-value“).

Frage: Wie lassen sich diese Vorgänge mittels einer Skizze erklären? Überlegen Sie sich eine Darstellung, welche Sie dabei unterstützt, das Ganze Unkundigen verständlich zu machen.

Überzeugen Sie sich, dass dieser Sachverhalt mit den Ausgaben Ihres Programms übereinstimmt!



Versuchen Sie die Abläufe rund um die Parameterübergabe bei Funktionen anhand einer Skizze zu veranschaulichen.

2.6 Zeichenketten

Die Sprache C kennt keinen Datentyp zur Speicherung von Zeichenketten (Folgen von Buchstaben). Auch hier werden die Pointer verwendet um das Problem zu lösen.

Die Zeichen (vom Typ `char`) werden in Arrays (oder sonst irgendwo im Speicher) hintereinander abgelegt und das Ende der Zeichenkette mit einem Nullbyte gekennzeichnet.

Die Standardbibliothek enthält zahlreiche Funktionen, die den Umgang mit solcherart definierten Zeichenketten vereinfachen. Tabelle 1 zeigt eine kleine Auswahl dieser Funktionen.

Element	Bedeutung
<code>"..."</code>	Literalkonstante für eine Zeichenkette. Das Nullbyte wird automatisch angefügt.
<code>strlen()</code>	Funktion zur Messung der Länge der Zeichenkette; das terminierende Nullbyte wird dabei nicht mitgezählt, obwohl es natürlich Speicherplatz benötigt!
<code>printf("%s", ...)</code> <code>scanf("%s", ziel)</code>	Ausgabe einer nullterminierten Zeichenkette Einlesen einer Zeichenkette von der Tastatur; Der Pointer ziel muss auf einen ausreichend großen Speicherbereich zeigen. Die terminierende Null wird automatisch in Ziel geschrieben. Der Lesevorgang bricht beim ersten Leerzeichen ab.
<code>strcpy(ziel, quelle)</code>	Kopie einer nullterminierten Zeichenkette; Der Pointer ziel muss auf einen ausreichend großen Speicherbereich zeigen. Die terminierende Null wird mitkopiert.

Tabelle 1: Einige gängige Funktionen zum Umgang mit Zeichenketten in C.

Erweitern Sie Ihr Programm um die Zeichenkettenfunktionen aus Tabelle 1 zu testen:

Lesen Sie eine Zeichenkette von der Tastatur ein, kopieren Sie sie in eine zweite Zeichenkette und kehren Sie die Zeichen in der Kopie um.

Geben Sie dann beide Zeichenketten aus.

Hinweis: Die Speicherbereiche für die Zeichenketten erzeugen Sie am Besten als Arrays mit Basistyp **char**.



Experimentieren Sie mit den Zeichenkettenfunktionen der C Standardbibliothek.

3 Dateizugriffe

In dieser Teilaufgabe wenden Sie die erarbeiteten Erkenntnisse an, indem Sie ein Programm zur Entschlüsselung (bzw. Verschlüsselung) von Textdateien schreiben.

3.1 Der „Verschlüsselungs“-Algorithmus

Gegeben sind ein Text in einer Datei und ein Schlüssel als Zeichenkette. Um den Text mit dem Schlüssel zu verschlüsseln, werden die beiden

³Der Stack ist ein Speicherbereich für die Versorgung von Unterprogrammen mit Parametern und lokalen Variablen, sowie die Speicherung der Rückkehradresse. Hier werden die neuesten Daten immer als erstes wieder gelöscht.

Zeichenketten durch die bitweise exklusiv-oder Operation (XOR) miteinander verknüpft⁴.

Frage: Machen Sie sich bewusst, wie die XOR-Verknüpfung definiert ist. Was passiert, wenn man einen beliebigen Wert zweimal mit dem selben Wert XOR-verknüpft?

Ist der Schlüssel kürzer als der Text, so wird der Schlüssel je nach Bedarf mehrfach wiederholt.

Wenn der Text **Steuerzeichen** (also Zeichen mit einem ASCII-Wert kleiner dem des Leerzeichens) enthält, dann werden diese **nicht verändert**. Das entsprechende Zeichen aus dem Schlüssel wird dabei **ungeutzt übersprungen**.

Damit die verschlüsselte Zeichenkette wieder als normaler Text behandelt (und z.B. auf dem Bildschirm ausgegeben) werden kann, sollte sie keine Steuerzeichen wie Zeilenvorschub, Seitenvorschub etc. sondern ausschließlich druckbare ASCII-Zeichen enthalten.

Um also zu vermeiden, dass ein druckbares Zeichen durch die Verschlüsselung in den Bereich der Sonderzeichen gerät, soll die exklusiv-ODER Operation mit einer geeigneten Maske auf die **niederwertigen vier Bit beschränkt** werden. Die höherwertigen vier Bit sollen also bei der Ver- bzw. Entschlüsselung immer unverändert bleiben.

Überzeugen Sie sich anhand der ASCII-Tabelle in Abbildung 3 davon, dass man durch Änderung der niederwertigen 4 Bit eines Zeichens nicht die Spalte wechselt und damit auch nicht von druckbaren Zeichen in die Sonderzeichen der ersten zwei Spalten geraten kann⁵!

Alle Zeichen mit ASCII-Werten kleiner als dem des Leerzeichens sollen durch die Verschlüsselung unverändert bleiben.

Um einen verschlüsselten Text wieder zu entschlüsseln, ist das Verschlüsselungsverfahren (mit dem selben Schlüssel, natürlich) erneut anzuwenden.

3.2 Arbeitsauftrag

Schreiben Sie eine Funktion mit zwei Argumenten vom Typ **char***, nämlich um die Zeichenkette für den Text, sowie für den Schlüssel übergeben zu können. Die Funktion soll den übergebenen Text, wie oben beschrieben, Zeichen für Zeichen mit dem übergebenen Schlüssel verschlüsseln.



Schreiben Sie eine Funktion zum Ver- bzw. Entschlüsseln einer Zeichenkette.

⁴Diese Art der „Verschlüsselung“, insbesondere in unserer eingeschränkten Umsetzung und wenn der Schlüssel kürzer ist als der Text oder gar mehrfach verwendet wird, ist natürlich nicht sicher und sehr leicht knackbar. „Echte“ Verschlüsselungen sind aber viel aufwendiger in der Umsetzung.

⁵Es kann immer noch das Steuerzeichen DEL entstehen wie aus der Tabelle ersichtlich ist. Diese Ausnahme wird jedoch toleriert, um den Algorithmus einfach zu halten.

	0	1	2	3	4	5	6	7
0	NUL 0	DLE 16	32	0 48	@ 64	P 80	` 96	p 112
1	SOH 1	DC1 17	! 33	1 49	A 65	Q 81	a 97	q 113
2	STX 2	DC2 18	" 34	2 50	B 66	R 82	b 98	r 114
3	ETX 3	DC3 19	# 35	3 51	C 67	S 83	c 99	s 115
4	EOT 4	DC4 20	\$ 36	4 52	D 68	T 84	d 100	t 116
5	ENQ 5	NAK 21	% 37	5 53	E 69	U 85	e 101	u 117
6	ACK 6	SYN 22	& 38	6 54	F 70	V 86	f 102	v 118
7	BEL 7	ETB 23	' 39	7 55	G 71	W 87	g 103	w 119
8	BS 8	CAN 24	(40	8 56	H 72	X 88	h 104	x 120
9	HT 9	EM 25	) 41	9 57	I 73	Y 89	i 105	y 121
A	LF 10	SUB 26	* 42	: 58	J 74	Z 90	j 106	z 122
B	VT 11	ESC 27	+ 43	; 59	K 75	[91	k 107	{ 123
C	FF 12	FS 28	, 44	< 60	L 76	\ 92	l 108	124
D	CR 13	GS 29	- 45	= 61	M 77	] 93	m 109	} 125
E	SO 14	RS 30	. 46	> 62	N 78	^ 94	n 110	~ 126
F	SI 15	US 31	/ 47	? 63	O 79	_ 95	o 111	DEL 127

Abbildung 3: ASCII-Tabelle

Testen Sie Ihre Implementierung, indem Sie die Funktion mit verschiedenen Eingabezeichenketten aufrufen.

Hinweis: Beachten Sie dabei insbesondere auch den Fall, dass der Schlüssel kürzer ist als der Text. Prüfen Sie auch, ob die Entschlüsselung Ihres verschlüsselten Textes funktioniert.

Ein verschlüsselter Text kann durch erneute „Verschlüsselung“ wieder in Klartext übersetzt werden. Falls also Ihre Funktion korrekt funktioniert, muss ein Text nach zweifachem Aufruf wieder als unveränderter Klartext vorliegen.

Um eine komplette Datei zu entschlüsseln, benötigen Sie eine weitere Funktion.

Diese muss mit Hilfe der Funktion `fopen()` die Eingabedatei und die Ausgabedatei öffnen und in einer Schleife die Zeilen der Eingabedatei jeweils mit `fgets()` lesen, entschlüsseln und mittels `fwrite()` in die Ausgabedatei schreiben.

Hinweis: Bei jeder neuen Zeile der Datei soll erneut mit dem ersten Zeichen des Schlüssels begonnen werden.

Wichtig: Verwenden Sie zum Schreiben von Texten, die möglicherweise nicht korrekt ausmaskierte Prozentzeichen (nämlich durch ein zweites Prozentzeichen) enthalten, keinesfalls die Funktion `printf` mit nur einem Parameter.

Die Funktion würde sonst versuchen, die Prozentzeichen als Formatanweisungen zu interpretieren und durch nicht vorhandene Parameter zu ersetzen - mit fatalen Folgen für die Zeichenkette und vielleicht auch das gesamte Programm.



Schreiben Sie eine Funktion, welche eine mehrzeilige Textdatei entschlüsseln kann.

Beachten Sie, dass das Ende der Eingabedatei erreicht ist, wenn der Aufruf von `fgets()` den Nullpointer als Rückgabewert liefert.

Entschlüsseln Sie nun mit Hilfe Ihrer Funktionen die Datei `??_geheim.txt` (siehe Kursportal) mit dem Passwort

Erdbeermilch

und schreiben Sie das Ergebnis in eine neue Datei namens `entschluesselt.txt`.



Schreiben Sie nun ein Programm, das es gestattet, Dateien zu verschlüsseln und zu entschlüsseln.